Tutorial 14A: Building a Simple RNN for Named Entity Recognition (NER)

Laiba Niazi

536672

1. Executive Summary

The objective was to develop a deep learning model using PyTorch to perform NER. By processing sequences of words, the model learns to assign tags such as **PERSON**, **LOCATION**, and **ORGANIZATION** to individual tokens. The project demonstrates how increasing training duration and adjusting hyperparameters directly impact the model's ability to generalize.

2. Data Preparation & Engineering

Before training, the raw text was converted into a format suitable for neural networks. This involved creating a vocabulary where every unique word is mapped to an integer.

- **Handling Variability:** Since sentences vary in length, <PAD> tokens were used to ensure uniform tensor shapes.
- **Unknown Tokens:** The <UNK> token was integrated to handle words not seen during the training phase.

```python
X_tensor = torch.tensor(X_padded, dtype=torch.long)
y_tensor = torch.tensor(y_padded, dtype=torch.long)

print("Input shape:", X_tensor.shape)
print("Target shape:", y_tensor.shape)
```

Python

```
Input shape: torch.Size([2, 6])
Target shape: torch.Size([2, 6])
```

3. The RNN Architecture

The model architecture defined in **Tutorial 14A.ipynb** consists of four primary stages:

- **Embedding Layer:** Maps word indices to a 50-dimensional dense vector space.
- **RNN Layer:** A recurrent layer with 50 (later increased to 128) hidden units to process the sequence.
- **Dropout:** A 10% dropout rate to mitigate overfitting.

- **Linear Head:** A final layer to map the RNN's hidden states to the total number of entity classes.

4. Comparative Training Analysis

The project highlights the necessity of iterative refinement.

Initial Attempt (3 Epochs)

With only 3 training cycles, the model failed to learn the distinction between classes, often defaulting to a single label for every word.

```
Training RNN Model
Epoch 1/3 - Loss: 1.5421
Epoch 2/3 - Loss: 1.5037
Epoch 3/3 - Loss: 1.4291
```

```
Word: Barack     Predicted Label: PERSON
Word: Obama      Predicted Label: PERSON
Word: went       Predicted Label: PERSON
Word: to         Predicted Label: PERSON
Word: Hawaii     Predicted Label: <PAD>
```

Optimized Attempt (100 Epochs)

By increasing the epochs to 100 and utilizing a learning rate of 0.005, the loss significantly decreased from ~1.54 to ~0.002.

```
model_improved = SimpleRNN_NER(vocab_size, embedding_dim, rnn_units, num_classes)

# We use the same loss function, but increase the learning rate slightly
optimizer_improved = optim.Adam(model_improved.parameters(), lr=0.005)

epochs = 100
```

```
Training Improved Model (100 Epochs)...
Epoch 20/100 - Loss: 0.0234
Epoch 40/100 - Loss: 0.0051
Epoch 60/100 - Loss: 0.0017
Epoch 80/100 - Loss: 0.0015
Epoch 100/100 - Loss: 0.0020
```

```
Word: Barack     Predicted Label: PERSON
Word: Obama      Predicted Label: PERSON
Word: went       Predicted Label: O
Word: to         Predicted Label: O
Word: Hawaii     Predicted Label: LOCATION
```

5. Model Validation & Results

To verify the model's accuracy, it was tested against a custom sentence: *"I was born in Pakistan"*.

```
# Test with Custom Data
custom_sentence = ["I was born in Pakistan"]
```

- **Performance:** The model successfully identified "Pakistan" as a **LOCATION**.
- **Inference Logic:** During inference, the model uses torch.no_grad() to save memory and torch.argmax() to select the most likely label for each word.

```
Custom Sentence Predictions:
Word: I          Predicted Label: O
Word: was        Predicted Label: O
Word: born       Predicted Label: O
Word: in         Predicted Label: O
Word: Pakistan   Predicted Label: LOCATION
```

6. Conclusion

The project successfully demonstrates that a **Simple RNN** can effectively solve the NER task when provided with sufficient training time and a well-defined embedding space. Key to this success was the use of ignore_index in the loss function, which allowed the model to focus exclusively on meaningful text while ignoring structural padding.